Объектно-ориентированное программирование

6 семестр
Лекция 8.

Содержание лекции:

- Чистая архитектура
- Пример приложения
- Docker, Swagger, EF, Миграции

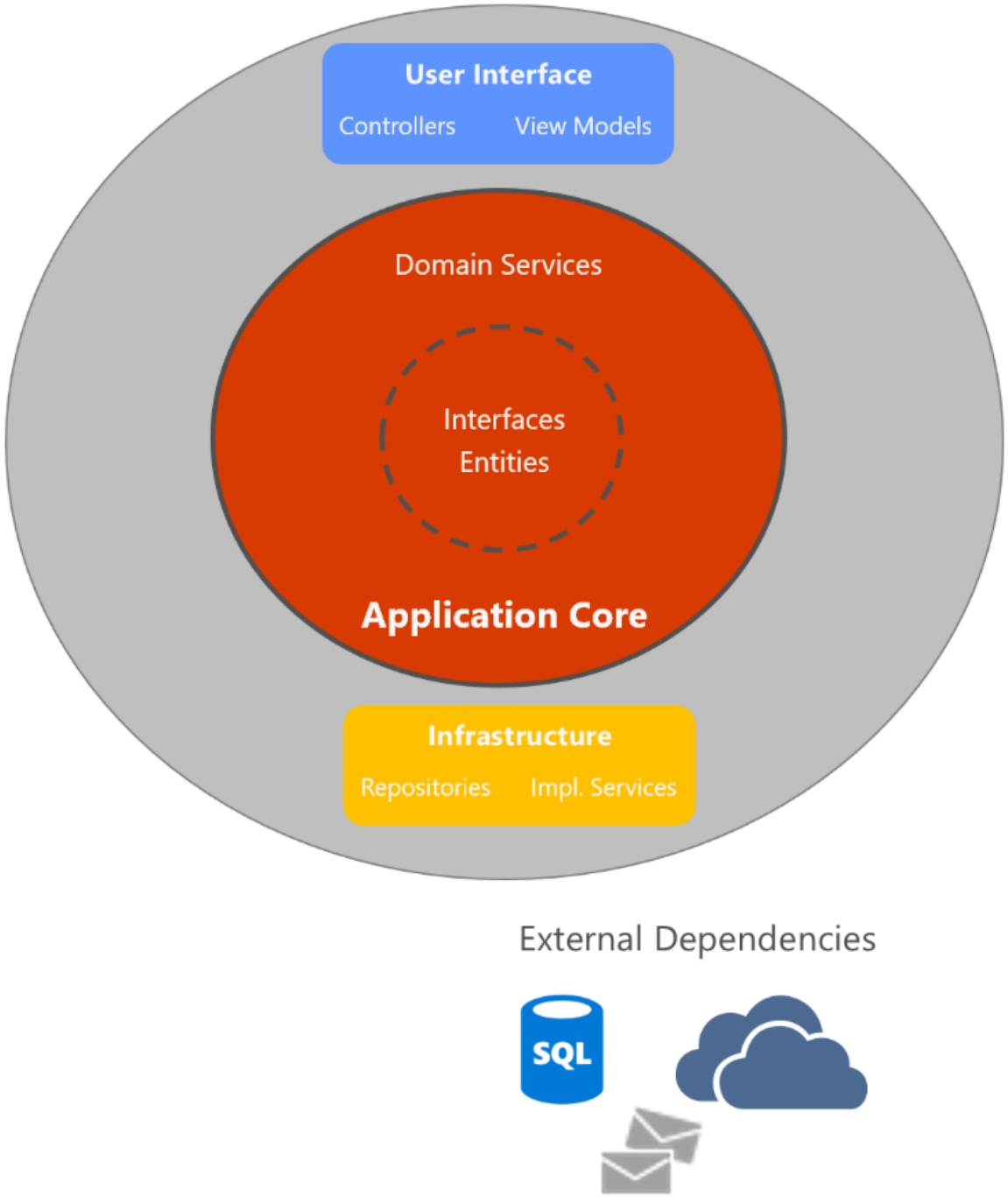
Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Чистая архитектура

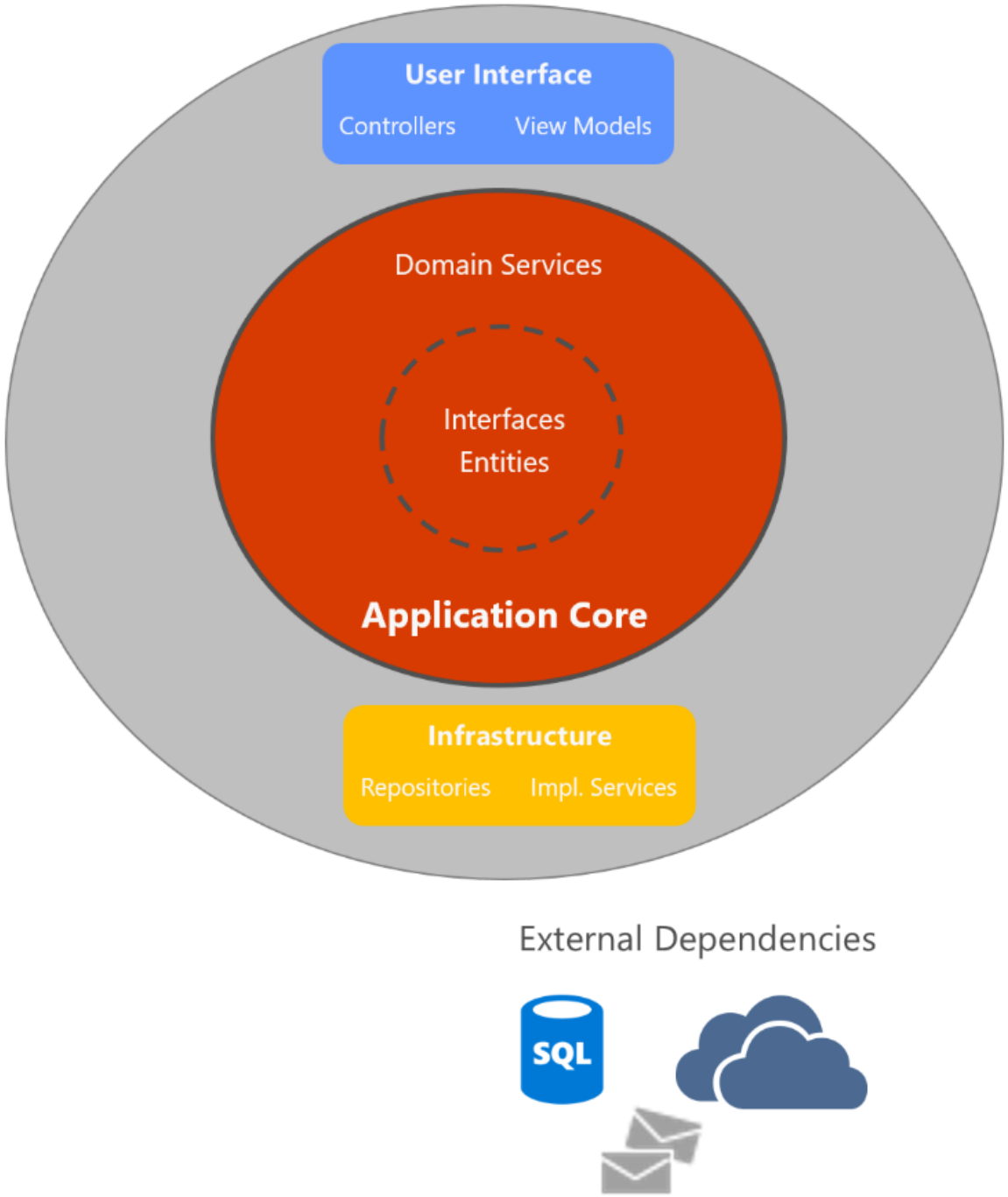
Роберт Сесил Мартин. Чистая архитектура. Искусство разработки программного обеспечения. 2017 г. (публикация 1-й статьи в 2012 г.)

"Если вы считаете, что хорошая архитектура дорого, попробуйте плохую архитектуру". - *Брайан Колон и Джозеф Yoder*

В рамках чистой архитектуры центральным элементом приложения являются его бизнес-логика и модель. В этом случае бизнес-логика не зависит от доступа к данным или другим инфраструктурам, то есть стандартная зависимость инвертируется: инфраструктура и детали реализации зависят от ядра приложения.



Эта функциональность достигается путем определения абстракций или интерфейсов в ядре приложения, которые реализуются типами, определенными в слое инфраструктуры. Такую архитектуру обычно рисуют в виде серии окружностей с общим центром, которая внешне напоминает срез луковицы.



Чистая архитектура

Чистую архитектуру можно описать фразой **- строй приложение вокруг бизнес-логики.**

Entities

- Сущности, которые содержат самую важную логику/алгоритмы
- **Пример:** расчет угла броска снаряда

Use Cases

- Наши бизнес-процессы
- Этот слой содержит прикладную логику - логику нашей предметной области
- Занимается тем, что манипулирует Entities
- **Пример:** приложения для расчета угла удара в бильярде и траектории выстрела из пушки

Контроллеры/Презентеры

- Те, с кем внешний мир взаимодействует
- **Пример:** MVC контроллеры, заглушка для HTTP, модель представления (MVVM)

UI, БД, Устройства

- Это тот самый внешний мир
- **Пример:** база данных, внешний сервис кеширования или логирования

При правильной реализации поток управления выглядит так:

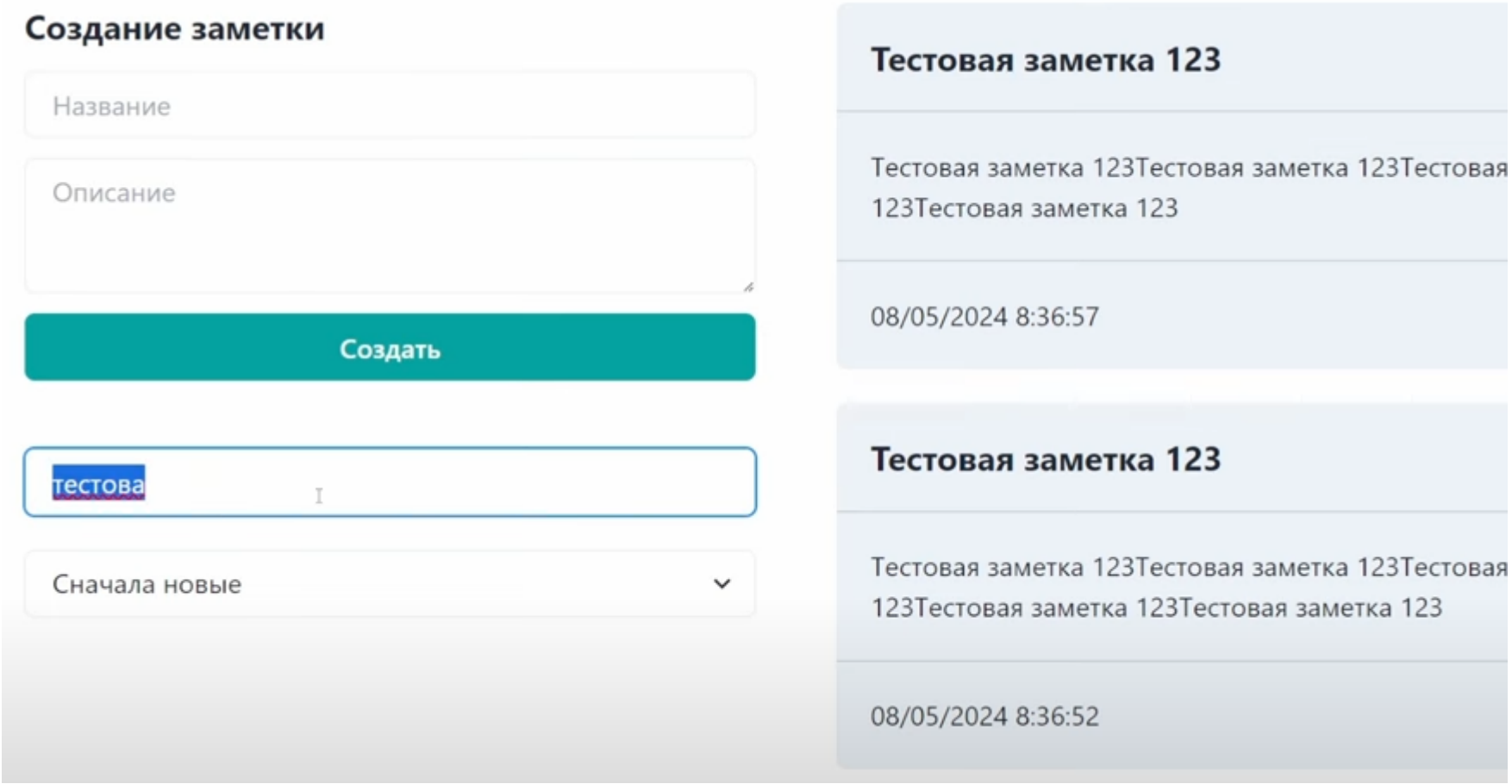
Controller -> Use Case Interactor -> Presenter.

Пример:

1. Клиент сделал запрос на контроллер;
2. Контроллер вызвал нужный метод на доменном сервисе;
3. Доменный сервис вызвал метод хранилища, который обновил базу данных.

Пример:

Приложение для составления заметок.



Структура проекта (папок):

Notes_fullstack_project

- NotesBackend
  - Core
    - Domain
    - Application
  - Api
  - Infrastructure
    - Persistence
    - другие внешние фреймворки (Kafka / RabbitMQ, Reddis)
- NotesFrontend

Проектирование классов

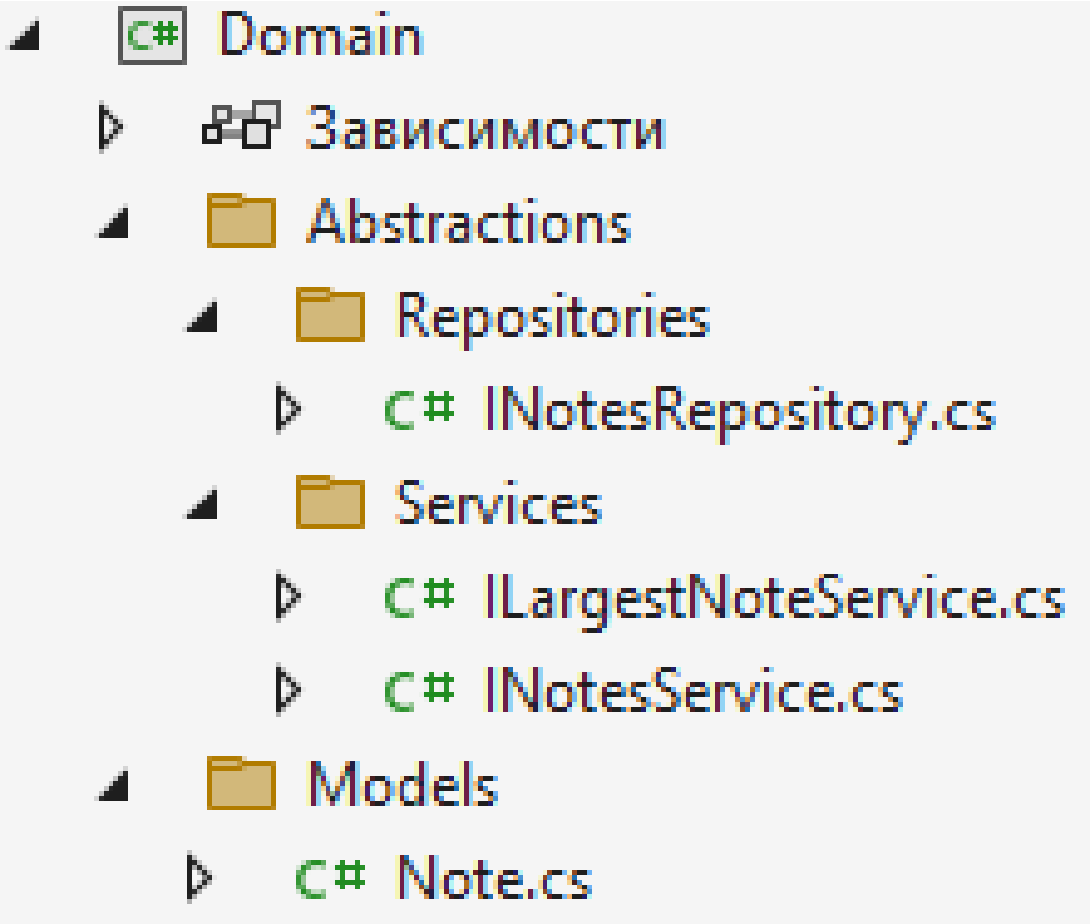
Важно:

- Необходимо начинать с Домена (бизнес модели и логики)
- Внутренние слои не знают о внешних
- Внутренний слой общается с внешними слоями через инверсию зависимости (через интерфейсы)
- Внешние слои знают о внутренних
- Обращение через слой не хорошо, но допустимо

Domain, модель Note

```
public class Note
{
    public const int MAX_TITLE_LENGTH = 256;
// Guid – Global unique id
    public Note(Guid id, string title, string description,
                DateTime createdAt)
    {
        if (title.Length > MAX_TITLE_LENGTH)
            throw new ArgumentException($"Title mustn`t be bigger
than {MAX_TITLE_LENGTH}");

        Id = id;
        Title = title;
        Description = description;
        CreatedAt = createdAt;
    }
 // Продолжение далее
```

Domain




```
public Guid Id { get; }
public string Title { get; }
public string Description { get; }
public DateTime CreatedAt { get; }

} // конец Note
```

Rich (богатая) модель – модель с логикой (действиями).

В Domain принято создавать rich модели.

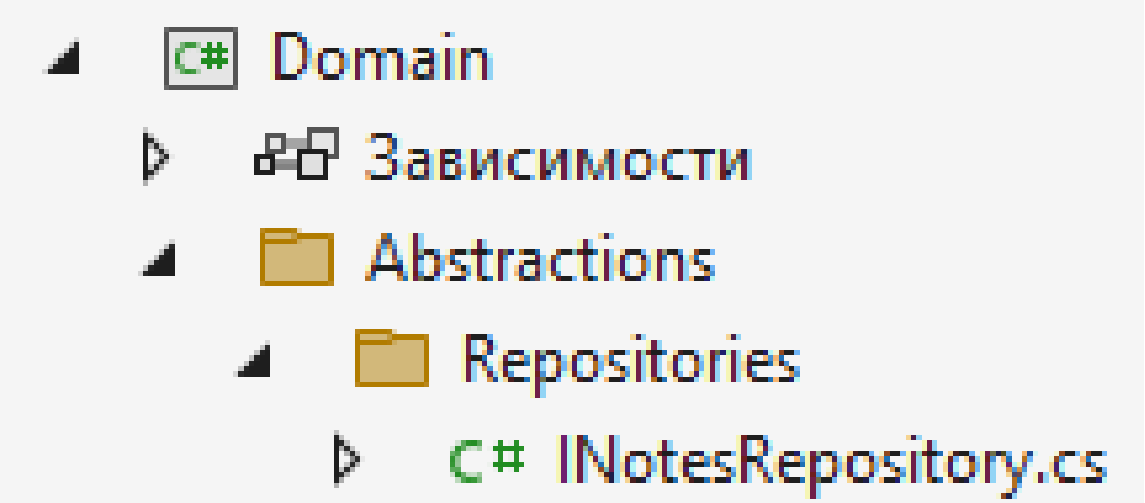
Противоположность – anemic (анемичная) модель – модель без логики.

Domain

Допустим, у нашей бизнес логики должна быть возможность получать, добавлять, изменять, удалять заметки (**CRUD** - операции).

Создадим интерфейс (**паттерн Репозиторий**)

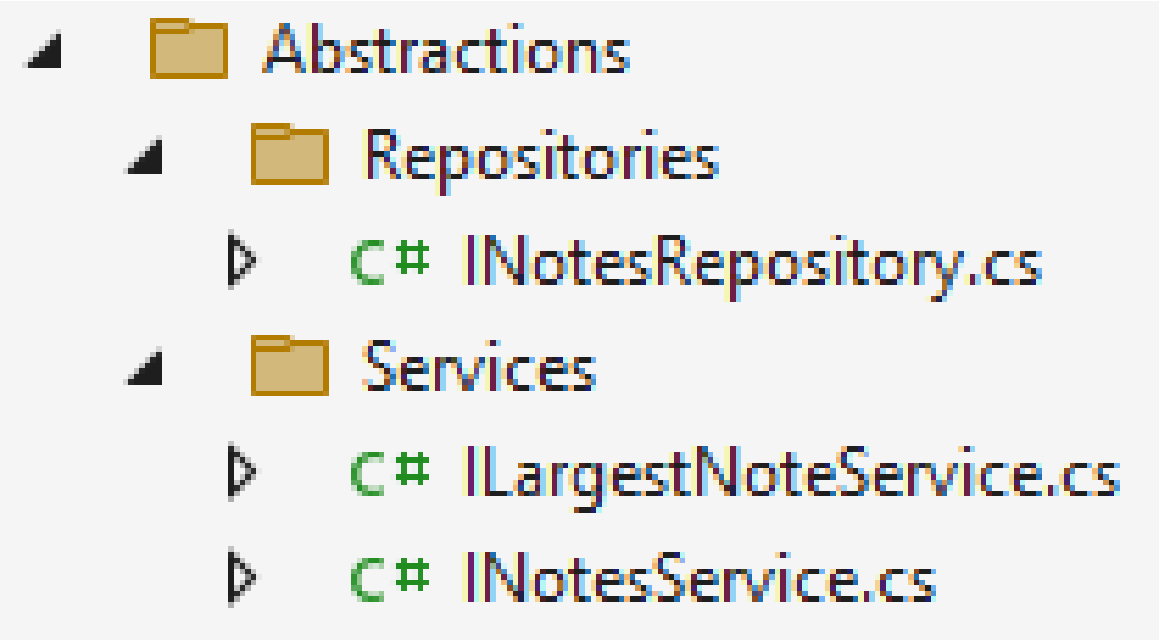
```
public interface INotesRepository
{
    Task<List<Note>> GetAll();
    Task<Note?> GetById(Guid id);

    Task<Note> Create(Note note);
    Task<Note> Update(Note note);
    Task<Guid> Delete(Guid id);
}
```



Domain

Также создадим интерфейсы для нужных нам сервисов, например, который предоставляет наибольшую заметку.

```
public interface ILargestNoteService
{
    Task<Note?> GetLargestNote();
}
```



Domain

Сервис по созданию, удалению и редактированию заметок.

```
public interface INotesService
{
    public Task<Note> CreateNote(string title,
                                 string description);

    public Task<Guid> DeleteNote(Guid id);

    public Task<Note> UpdateNote(Guid id,
                                 string? title,
                                 string? description);
}
```

Замечание

Сравним 2 метода и ответим на вопрос

«Какой метод лучше спроектирован?»

```
public Task<Note> CreateNote(string title,
                             string description);
```

ИЛИ

```
public record CreateNodeCommand(string title,
                                string description)

public Task<Note> CreateNote(CreateNodeCommand command);
```

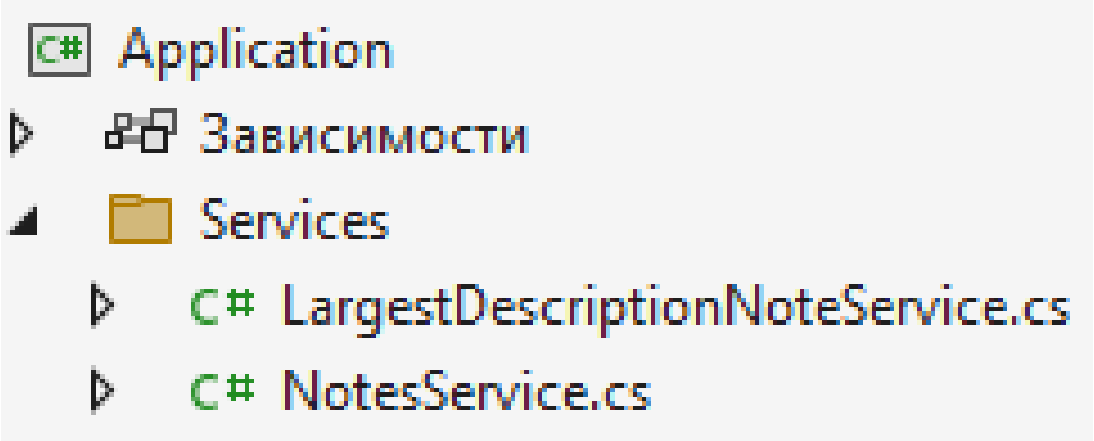
Второй вариант обеспечивает **гибкость при изменении требований.** Если в будущем потребуется добавить новые поля (например, tags, priority), в первом варианте придется менять сигнатуру метода, что может привести к поломке существующего кода. Во втором варианте достаточно просто добавить поле в CreateNodeCommand.

```
public record CreateNodeCommand(string title,
                                string description)

public Task<Note> CreateNote(CreateNodeCommand command);
```

Application (Use-cases) / Слой приложения (сценарии использования)

Реализуем интерфейсы сервисов.

Замечание: Здесь мы можем ссылаться только на Domain.




```
public class LargestDescriptionNoteService : ILargestNoteService
{
    private readonly INotesRepository _notesRepository;

// Используем принцип Inversion of Control (IoC), который
реализован в подходе Dependency Injection
    public LargestDescriptionNoteService(
                                INotesRepository notesRepository)
    {
        _notesRepository = notesRepository;
    }
```

```
public class LargestDescriptionNoteService : ILargestNoteService
{
    private readonly INotesRepository _notesRepository;

    public LargestDescriptionNoteService(
                                INotesRepository notesRepository)
    {
        _notesRepository = notesRepository;
    }

    public async Task<Note?> GetLargestNote()
    {
        var notes = await _notesRepository.GetAll();

        return notes.MaxBy(n => n.Description.Length);
    }
}
```

Application (Use-cases) / Слой приложения (сценарии использования)

Реализуем интерфейс `INotesService` в классе `NotesService`, который управляет заметками.

```
public class NotesService : INotesService
{
    private readonly INotesRepository _repository;

    public NotesService(INotesRepository repository)
    {
        _repository = repository;
    }
```

```
public async Task<Note> CreateNote(string title,
                                   string description)
{
    var note = new Note(Guid.NewGuid(),
                        title,
                        description,
                        DateTime.UtcNow);

    await _repository.Create(note);

    return note;
}

public async Task<Guid> DeleteNote(Guid id)
{
    return await _repository.Delete(id);
}
```

```
    public async Task<Note> UpdateNote(Guid id,
                                       string? title,
                                       string? description)
    {
        var note = await _repository.GetById(id);

        if (note == null)
            throw new Exception("Note not found");

        var updated = new Note(id,
                               title ?? note.Title,
                               description ?? note.Description,
                               DateTime.UtcNow);

        return await _repository.Update(updated);
    }
}
```

Infrastructure / Слой инфраструктуры

Замечание: Здесь мы можем ссылаться на Domain и Application.

В данном слое возможно обращение к БД (Dapper, Entity Framework), использование Фреймворков для кеширования (Reddis), логирования, брокеров сообщений (RabbitMQ, Kafka).

Проект **Persistence** отвечает за обращение к БД через ORM (Object-Relation Mapping) Entity Framework. Все остальные сервисы (Reddis, Kafka и т.п.) необходимо располагать в других отдельных проектах в слое **Infrastructure**.

Персистентность - это возможность хранить объекты постоянно, даже между выполнениями программы.

Принято разделять модели на 3 вида:

1. Domain Model (Доменная модель);
2. Entity (Модель БД);
3. DTO (Data Transfer Object – модель для передачи данных).

Замечание:

В небольших проектах без сложной бизнес-логики Domain и Entity модели зачастую объединяются.

1. Domain Model (Доменная модель)

Что это?

- Ядро бизнес-логики приложения.
- Содержит правила, валидацию и методы для работы с данными.

Характеристики:

✅ Инкапсуляция (приватные поля, методы)

✅ Бизнес-правила (например, Student.CanEnroll())

✅ Независимость от инфраструктуры (БД, API)

2. Entity (Модель БД)

Что это?

- Представление данных для работы с базой (EF Core).
- Содержит реляционные аннотации ([Key]) или Fluent API-конфигурации.

Характеристики:

✅ Публичные сеттеры (для ORM)

✅ Навигационные свойства (ICollection<Grade>)

✅ Атрибуты БД ([MaxLength], [ForeignKey])

3. DTO (Data Transfer Object)

Что это?

- Объект для передачи данных между слоями (API ↔ Frontend).
- Нет логики — только данные.

Характеристики:

✅ Плоская структура (без сложных связей)

✅ Оптимизирован для клиента (только нужные поля)

✅ Сериализуем в JSON/XML

Цепочка вызовов классов в жестком следовании чистой архитектуре:

1. Клиент отправляет запрос **CreateNoteRequest (DTO)** в наш API.
2. Контроллер принимает запрос и преобразует во внутренний контракт **CreateNoteCommand (DTO)** и передает команду в нужный сервис.
3. Сервис принимает команду, создает доменную модель **Note (Domain)** и передаёт в репозиторий для сохранения.
4. Репозиторий принимает доменную модель и создаёт модель БД **NoteEntity (Entity)**.

Гибкость приложения достигается за счёт ~~огромного количества одинаковых классов~~ разделения программы на слои и создания контрактов между этими слоями.

Плюсы разделения:

– Каждый класс живёт в своём слое,

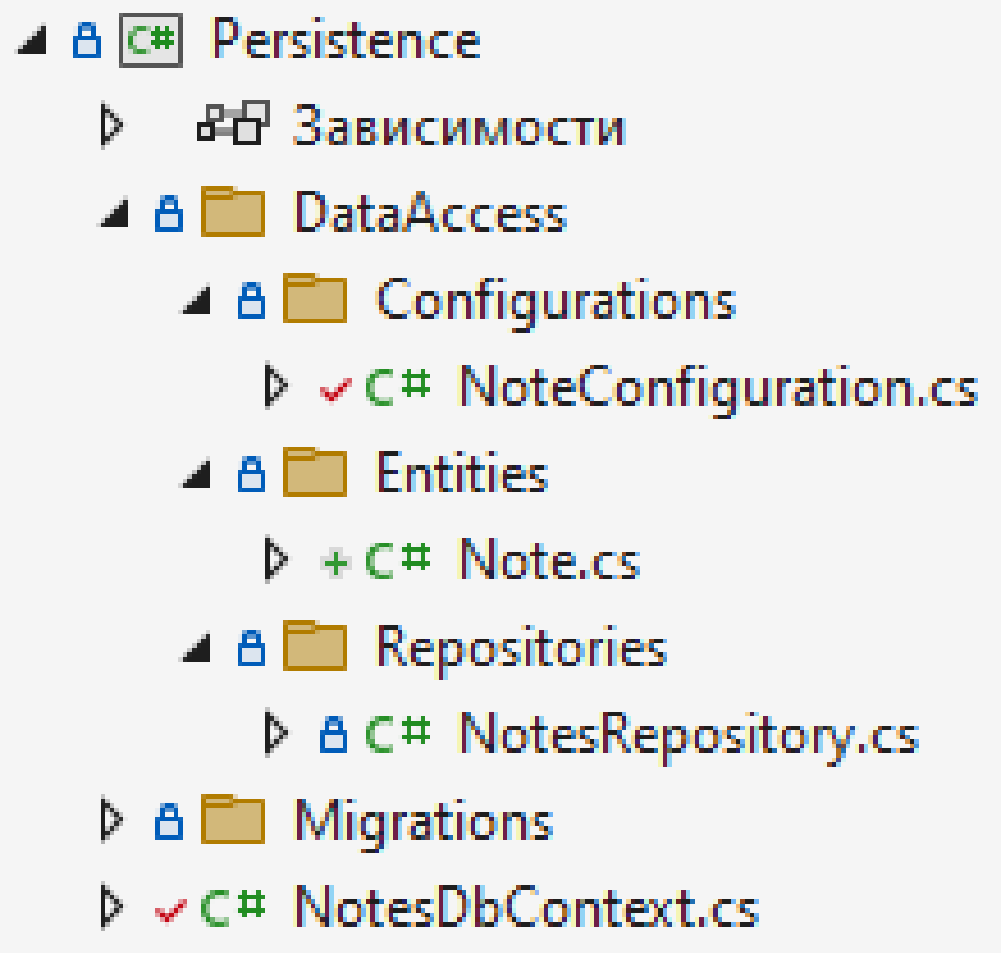
– Изменения в одном месте не вызывают “эффекта домино”,

– Система становится как Lego — собирается из независимых блоков.

В малых приложениях с целью ускорения разработки **Request** и **Command** объединяются, также, как и **Domain** и **Entity,** ценой масштабируемости.

Infrastructure. Persistence.

Рассмотрим структуру хранимых данных. Зачастую данные модели и БД не совпадают. Поэтому для хранения используются отдельные классы, которые называются **Entities**.




```
public class Note
{
    public Guid Id { get; set; }
    public string Title { get; set; } = null!;
    public string Description { get; set; } = null!;
    public DateTime CreatedAt { get; set; }
}
```

Infrastructure. Persistence.

Рассмотрим контекст данных.

```
public class NotesDbContext : DbContext
{
    public NotesDbContext(
                        DbContextOptions<NotesDbContext> options)
        : base(options)
    {}
     // Конфигурирование таблиц БД
    protected override void OnModelCreating(
                                    ModelBuilder modelBuilder)
    {// Вынесем конфигурацию заметки в отдельный класс
        modelBuilder.ApplyConfiguration(
                                    new NoteConfiguration());
    }
     // С помощью метода Set инициализируем коллекцию заметок
    public DbSet<Note> Notes => Set<Note>();
}
```

Конфигурирование таблицы заметок.

```
public class NoteConfiguration : IEntityTypeConfiguration<Note>
{
    public void Configure(EntityTypeBuilder<Note> builder)
    {
        builder.HasKey(x => x.Id);

        builder.Property(n => n.Title)
            .HasMaxLength(Note.MAX_TITLE_LENGTH)
            .IsRequired();
        builder.Property(n => n.Description)
            .IsRequired();
        builder.Property(n => n.CreatedAt)
            .IsRequired();
    } // По умолчанию конфигурация будет такой же, кроме
      // HasMaxLength(Note.MAX_TITLE_LENGTH)
} // IsRequired делает свойство необходим при добавлении заметки
```

Реализуем репозиторий

```
public class NotesRepository : INotesRepository
{
    private readonly NotesDbContext _context;

    public NotesRepository(NotesDbContext context)
    {
        _context = context;
    }
```

```
//Добавление заметки в БД
 public async Task<Note> Create(Note note)
 {
     var noteEntity = new Entities.Note(note.Id,
                                        note.Title,
                                        note.Description,
                                        note.CreatedAt);

     await _context.AddAsync(noteEntity);
     await _context.SaveChangesAsync();

     return note;
 }
```

```
//Получаем все заметки из БД
public async Task<List<Note>> GetAll()
{
    var notesEntities = await _context.Notes
        .AsNoTracking()
        .ToListAsync();
 // Чтобы данные не помещались в кэш, применяется метод
    AsNoTracking().
    var notes = notesEntities
        .Select(n => new Note(n.Id,
                              n.Title,
                              n.Description,
                              n.CreatedAt))
        .ToList();

    return notes;
}
```

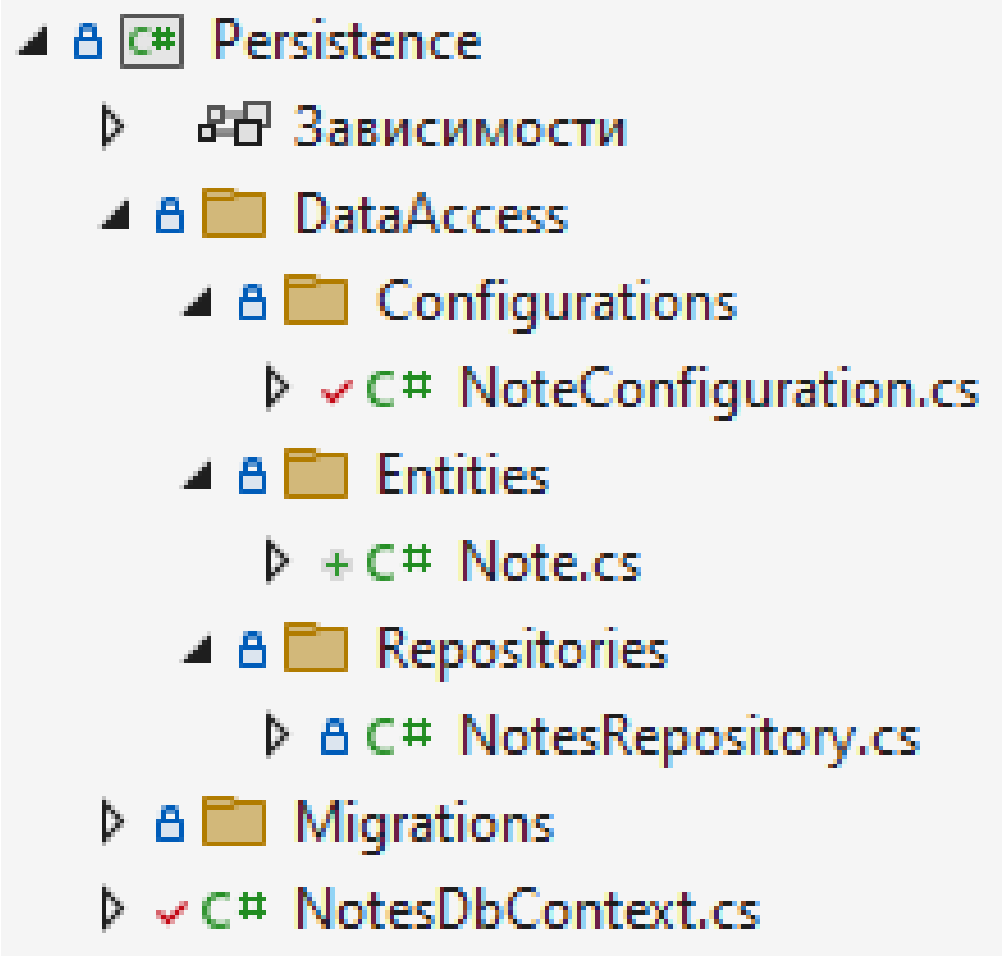
```
public async Task<Note?> GetById(Guid id)
{
    var noteEntity = await _context.Notes
        .AsNoTracking()
        .SingleOrDefaultAsync(n => n.Id == id);

// Если заметки есть и она 1, то мы её получим, иначе будет null

    if (noteEntity == null)
        return null;

    return new Note(noteEntity.Id,
                    noteEntity.Title,
                    noteEntity.Description,
                    noteEntity.CreatedAt);
}
```

```
public async Task<Note> Update(Note note)
{
    await _context.Notes
        .Where(n => n.Id == note.Id)
        .ExecuteUpdateAsync(s => s
            .SetProperty(n => n.Title, n => note.Title)
            .SetProperty(n => n.Description,
                         n => note.Description)
            .SetProperty(n => n.CreatedAt,
                         n => note.CreatedAt));

    return note;
}
```

```
    public async Task<Guid> Delete(Guid id)
    {
        await _context.Notes
            .Where(n => n.Id == id)
            .ExecuteDeleteAsync();

        return id;
    }
}
```

Infrastructure. Persistence.

Папка **Migrations** создаётся автоматически при создании миграций с помощью **Entity Framework**. Рассмотрим позже, когда будем настраивать проект.



Web

Рассмотрим последний слой – слой веб приложения. На наше **Web Api** будут приходит запросы (**request**) от клиентов (**frontend**), а наше **Api** должно обрабатывать их и отправлять ответы (**response**).

Наш **Web Api** состоит из компонентов:

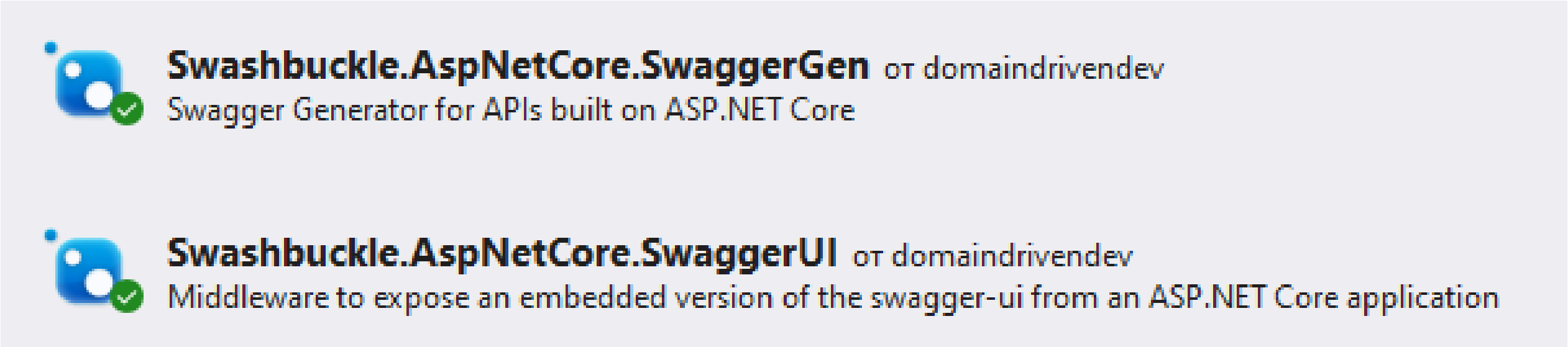
1. Точка входа **Program**, которая включает, подключение сервисов и конфигурирование
2. **Контроллеры**
3. **Контракты**
4. **Конфигурационные файлы**: appsettings.json, docker-compose.yml, *Properties*/launchSettings.json.

Web. Program. Подключаем сервисы. Т.к. мы используем контроллеры, то нужно написать `AddControllers();` альтернативно (или совместно) можно использовать **Minimal API**.

Подключаем **Swagger** - набор инструментов, который позволяет автоматически описывать API на основе его кода (отправлять и получать запросы в удобном UI).

```
public class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        builder.Services.AddSwaggerGen();
        builder.Services.AddControllers();
```

Скачиваем Swagger через менеджер пакетов NuGet



Web. Program

```
builder.Services.AddDbContext<NotesDbContext>(options =>
{
  options.UseNpgsql(
    builder.Configuration.GetConnectionString(
      nameof(NotesDbContext)));
});
```

nameof(NotesDbContext) выдаст строку "NotesDbContext".

Здесь мы явно указываем `UseNpgsql` – какую БД будем использовать – PostgreSQL.

Скачиваем Entity Framework через менеджер пакетов NuGet

Microsoft.EntityFrameworkCore от Microsoft 9.0.1
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite, Azure Cosmos DB, M...

Microsoft.EntityFrameworkCore.Design от Microsoft 9.0.1
Shared design-time components for Entity Framework Core tools.

Microsoft.EntityFrameworkCore.Relational от Microsoft 9.0.1
Shared Entity Framework Core components for relational database providers.

Microsoft.EntityFrameworkCore.Tools от Microsoft 9.0.1
Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

Npgsql.EntityFrameworkCore.PostgreSQL от Shay Rojansky,Austin Drenski,Yoh Deadfall 9.0.3
PostgreSQL/Npgsql provider for Entity Framework Core.

Web. Program

Cross-Origin Resource Sharing (CORS) «совместное использование ресурсов между разными источниками» — технология современных браузеров, которая позволяет предоставить веб-страницам доступ к ресурсам другого домена. Т.к. frontend предполагается писать на другом языке, то необходимо подключить CORS.

```
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.WithOrigins("http://localhost:7164");
        policy.AllowAnyHeader();
        policy.AllowAnyMethod();
    });
});
```

Web. Program

Настраиваем сервисы, каждому интерфейсу сопоставляем конкретную реализацию. Сервисы помещаются в DI Container (Dependency Injection Container) – контейнер инъекции зависимости. Контейнер автоматически передаёт нужную реализацию, когда необходимо.

```
builder.Services.AddScoped<ILargestNoteService,
                           LargestDescriptionNoteService>();
builder.Services.AddScoped<INotesRepository, NotesRepository>();
builder.Services.AddScoped<INotesService, NotesService>();
```

Web. Program

Если проект запускается в режиме разработки, то добавляем swagger.

Не забываем добавить маршрутизацию к действиям контроллеров.

```
        var app = builder.Build();

        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        app.MapControllers();

        app.Run();
    }
}
```

Web. Controllers

Указываем атрибут `ApiController` – что данный класс – контроллер. Атрибут `Route("[controller]")` задаёт общий маршрут для всех методов контроллера. В () скобках указан шаблон, который означает, что маршрут это название контроллера – Notes. Т.е. данный контроллер слушает запросы, вида https://домен/Notes

```
[ApiController]
[Route("[controller]")]
public class NotesController : ControllerBase
{
```

Web. Controllers

Подключаем сервисы.

```
private readonly INotesService _noteService;
private readonly ILargestNoteService _largestNoteService;
private readonly NotesDbContext _notesDbContext;

public NotesController(
    INotesService noteService,
    ILargestNoteService largestNoteService,
    NotesDbContext notesDbContext)
{
    _noteService = noteService;
    _largestNoteService = largestNoteService;
    _notesDbContext = notesDbContext;
}
```

Web. Controllers

Указываем маршрут, данный метод обрабатывает запросы по адресу:

https://домен/Notes/Create

Атрибут [FromBody] означает, что данные (request) поступают из тела **Post** запроса.

```
[HttpPost("Create")]
public async Task<IActionResult> Create(
                                [FromBody] CreateNoteRequest request)
{
   var note = await _noteService.CreateNote(request.Title,
                                            request.Description);

   return Ok(new CreateNoteResponse(note));
}
```

Web. Controllers

Пишем **endpoint** для **Get** запроса. Конечная точка или **endpoint** представляет некоторый код, который обрабатывает запрос.

```
[HttpGet]
public async Task<IActionResult> Get(
                                [FromQuery] GetNotesRequest request)
{
    var notesQuery = _notesDbContext.Notes
        .AsNoTracking();

    if (request.Search != null)
    {
        var lower = request.Search.ToLower();

        notesQuery = notesQuery
            .Where(n => n.Title.ToLower().Contains(lower));
    }//Продолжение следует далее
```

```
    Expression<Func<Entities.Note, object>> selectorKey =
        request.SortItem?.ToLower() switch
        {
            "date" => note => note.CreatedAt,
            "title" => note => note.Title,
            _ => note => note.Id
        };
    notesQuery = request.SortOrder == "desc"
        ? notesQuery.OrderByDescending(selectorKey)
        : notesQuery.OrderBy(selectorKey);

    var notes = await notesQuery
        .Select(n =>
         new Note(n.Id, n.Title, n.Description, n.CreatedAt))
        .ToListAsync();
    return Ok(new GetNotesResponse(notes));
}
```

Web. Controllers

Пишем **endpoint** для **Delete** запроса.

```
    [HttpDelete("Delete")]
    public async Task<IActionResult> Delete(
                                [FromBody] DeleteNoteRequest request)
    {
        var note = await _noteService.DeleteNote(request.Id);

        return Ok(new DeleteNoteResponse(note));
    }
}
```

Web. Controllers

Пишем **endpoint** для **Update** запроса.

```
[HttpPatch("Update")]
public async Task<IActionResult> Update(
                                [FromBody] UpdateNoteRequest request)
{
   var note = await _noteService.UpdateNote(request.Id,
                                            request.Title,
                                            request.Description);

   return Ok(new UpdateNoteResponse(note));
}
```

Web. Controllers

Пишем **endpoint** для **Get** запроса по адресу

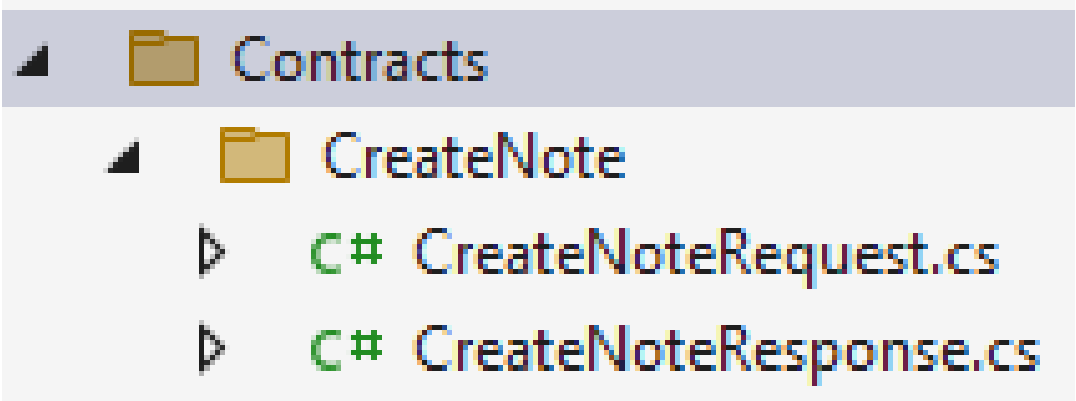
https://домен/Notes/LargestNote

```
    [HttpGet("LargestNote")]
    public async Task<IActionResult> GetLargestNote()
    {
        var largestNote = await
                                _largestNoteService.GetLargestNote();

        if (largestNote == null)
        {
            return Ok("No notes");
        }

        return Ok(new GetLargestNoteResponse(largestNote));
    }
}
```

Web. Contracts

Каждый запрос и ответ поместим в контракты и создадим record.

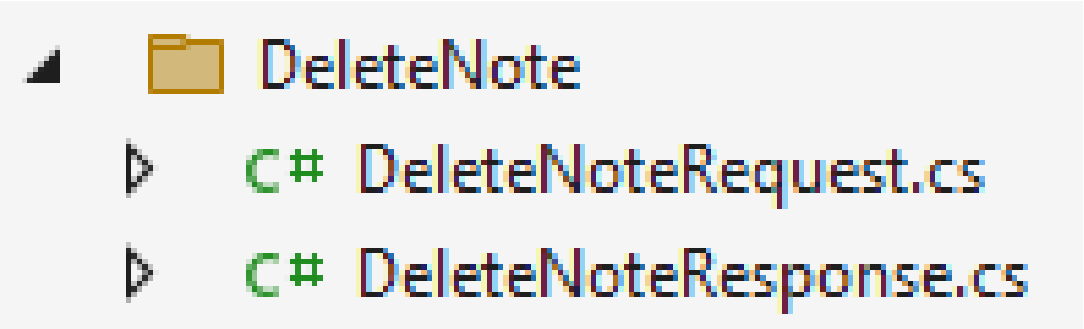
Запрос и ответ для создания заметки.




```
public record CreateNoteRequest(
    string Title,
    string Description);

public record CreateNoteResponse(Note Note);
```

Web. Contracts

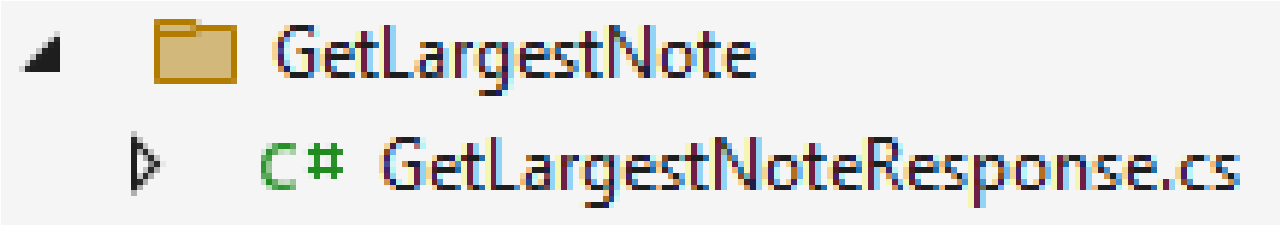
Запрос и ответ для удаления заметки.




```
public record DeleteNoteRequest(Guid Id);
public record DeleteNoteResponse(Guid Id);
```

Web. Contracts

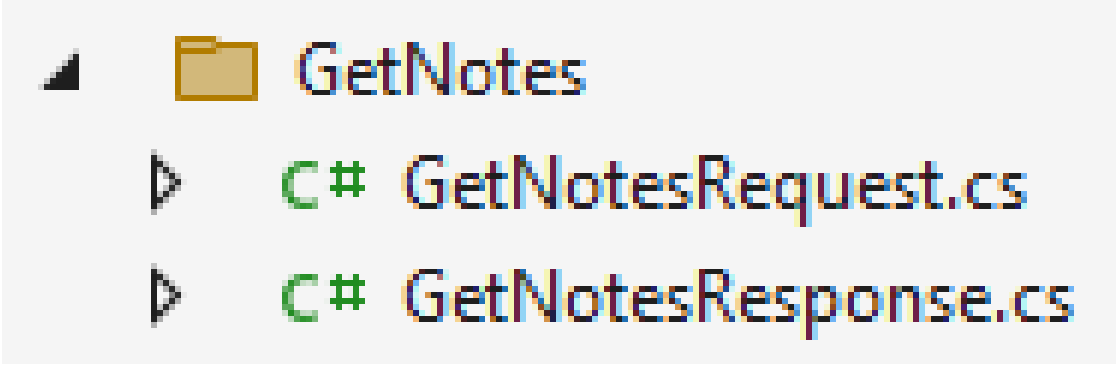
Ответ для самой большой заметки.




```
public record GetLargestNoteResponse(Note Notes);
```

Web. Contracts

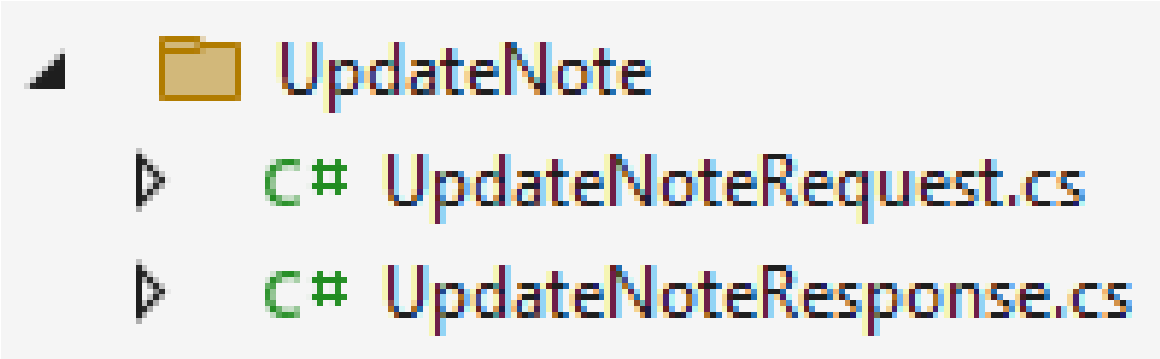
Запрос и ответ для получения заметок.




```
public record GetNotesRequest(
    string? Search,
    string? SortItem,
    string? SortOrder);

public record GetNotesResponse(List<Note> Notes);
```

Web. Contracts

Запрос и ответ для изменение заметок.




```
public record UpdateNoteRequest(
    Guid Id,
    string? Title,
    string? Description);

public record UpdateNoteResponse(Note Note);
```

Web. Конфигурационные файлы

appsettings.json

Настроим строку подключения к БД. Зайдем в файл appsettings.json и добавим:

```
 "ConnectionStrings": {
   "NotesDbContext": "Server=localhost;
                      Port=5433;
                      Database=my_notes;
                      User Id=postgres;
                      Password=postgres;"
 },
```

`Server=localhost` – адрес сервера.

`Port=5433` – любой, главное, чтобы не был занят и в докере был такой же.

`Database=my_notes` – имя БД, любое.

`User Id=postgres` – имя пользователя, по умолчанию.

`Password=postgres` – пароль, по умолчанию.

Web. Конфигурационные файлы

Проверим конфигурацию http. Зайдем в Properties/launchSettings.json

Во всех частях программы адрес должен быть одинаковый.

```
"applicationUrl": "https://localhost:7164"
```

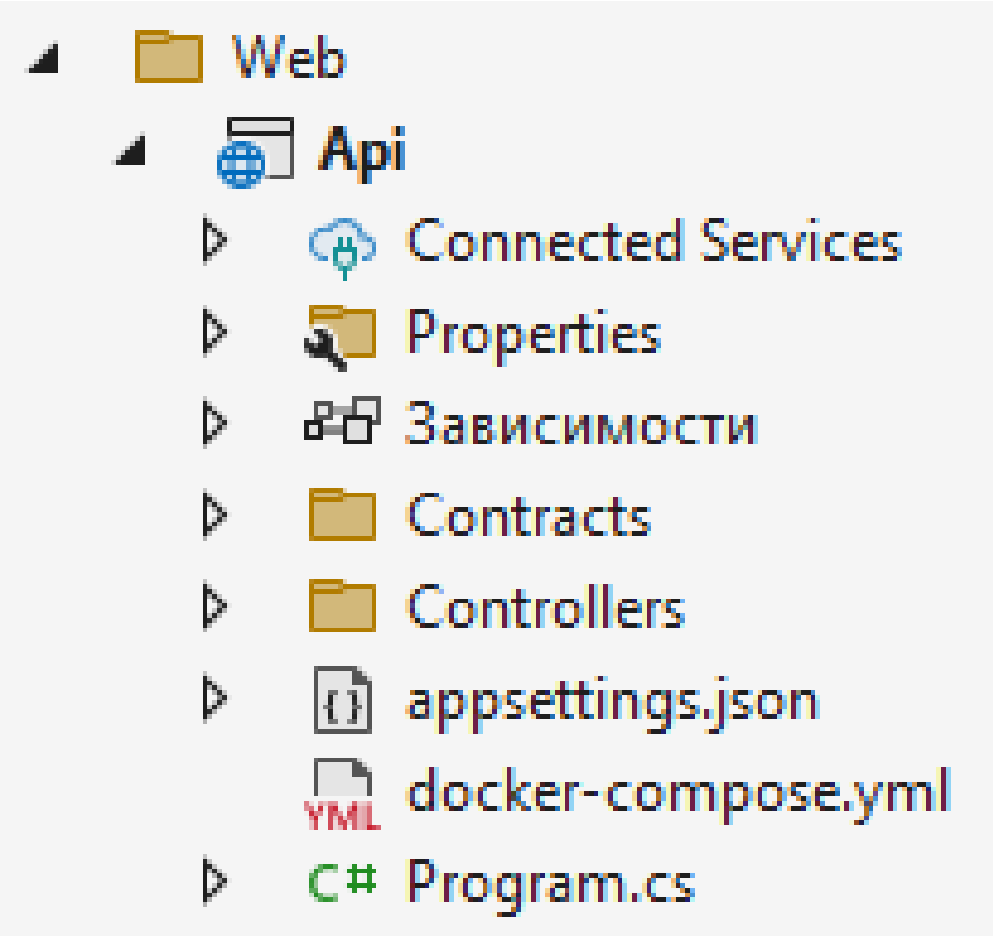
Порт любой, главное, чтобы не был занят.

```
 "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7164",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
```

Web. Конфигурационные файлы

Создадим докер файл. Назовём docker-compose.yml

Docker — это платформа с открытым исходным кодом для автоматизации разработки, доставки и развертывания приложений. Ее основная идея — создание стандартного и предсказуемого окружения, где приложения могут работать независимо от операционной системы или инфраструктуры.



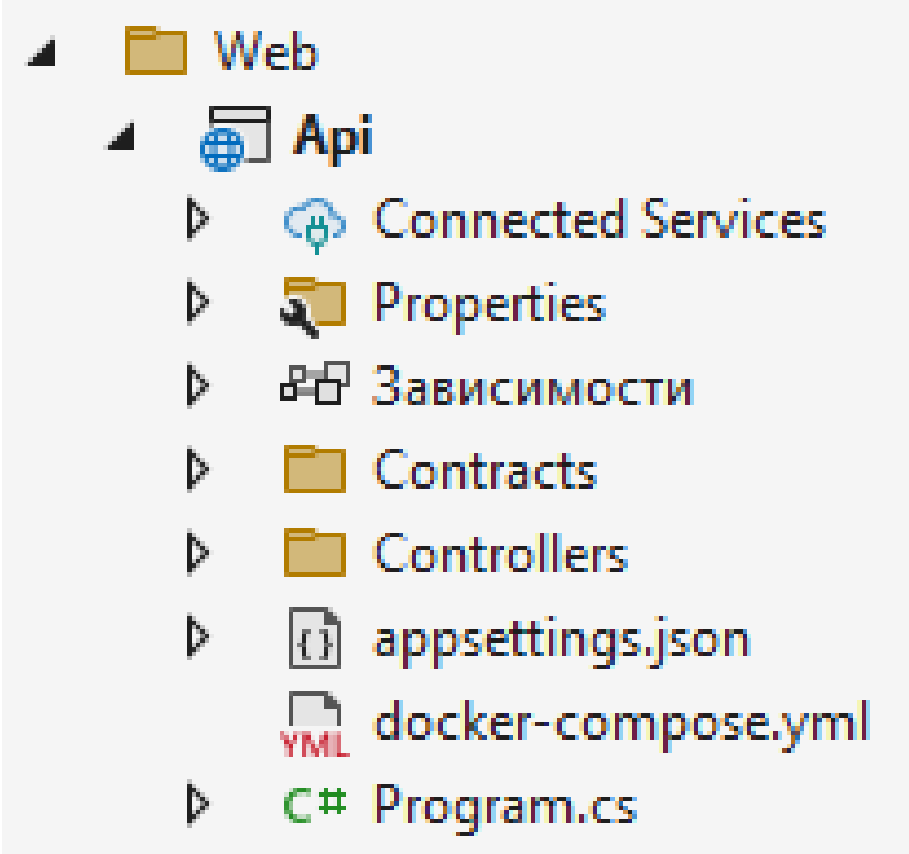
Docker широко применяется в сферах разработки ПО, **DevOps** и управления IT-инфраструктурой. Благодаря нему можно ускорить разработку и упростить перенос приложений между окружениями.

Сценарии использования:

- упаковывание вашего приложения (и так же используемых компонент) в docker контейнеры;
- раздача и доставка этих контейнеров вашим командам для разработки и тестирования;
- выкладывания этих контейнеров на ваши продакшены, как в дата центры так и в облака.

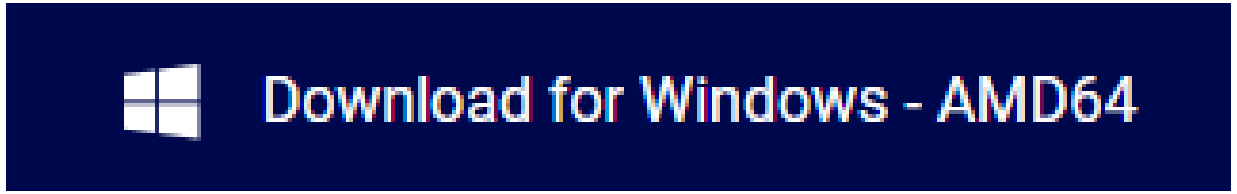
Web. Конфигурационные файлы

```yaml
services:
  postgres:
    container_name: postgres
    image: postgres:latest
    restart: always
    environment:
        POSTGRES_DB: my_notes
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
    volumes:
     - postgres-data:/var/lib/postgresql/data
    ports:
     - 5433:5432

volumes:
    postgres-data:
```



Переходим на сайт Docker и скачиваем Docker Desktop.

https://www.docker.com/get-started/

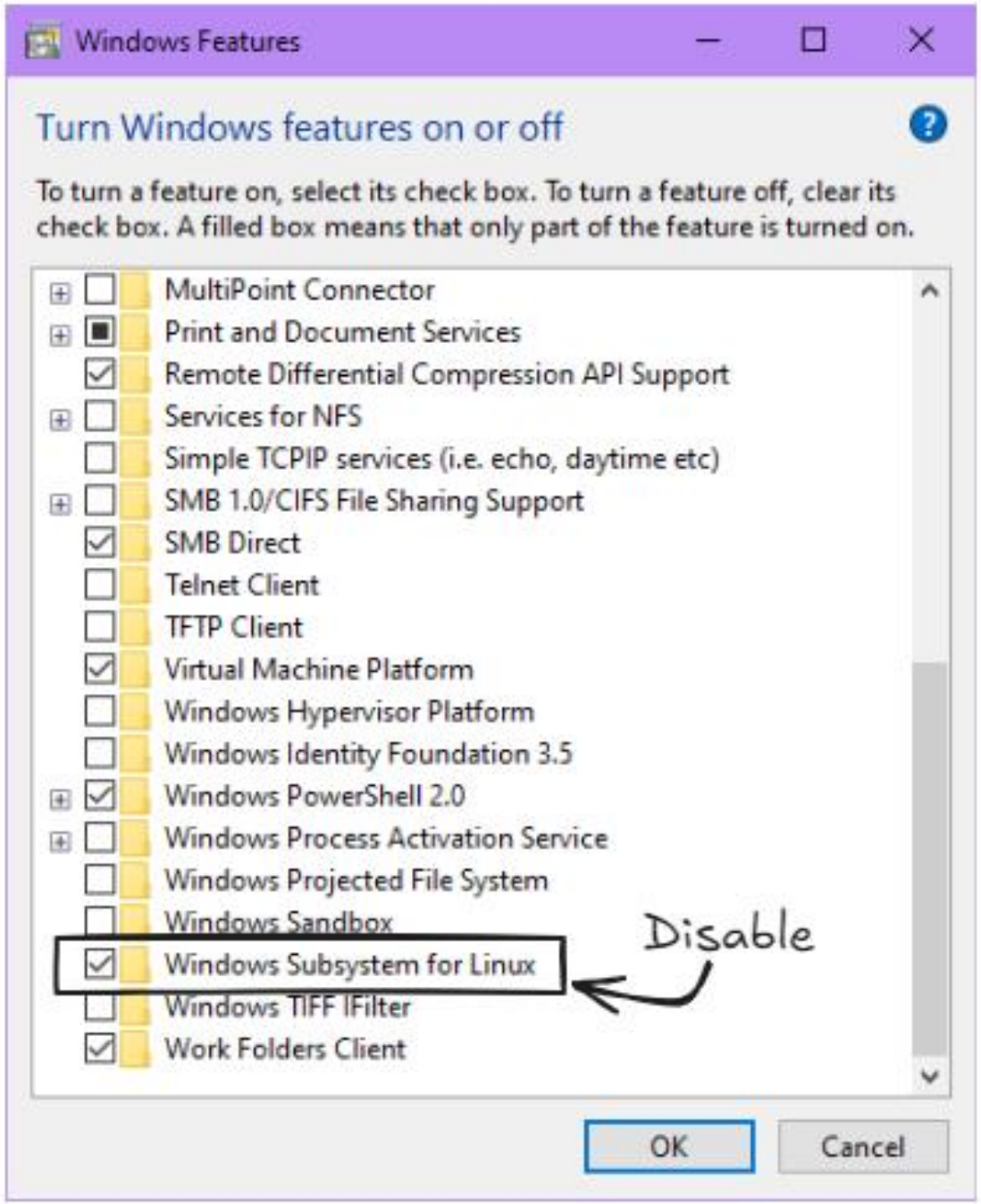


Запускаем докер. Если движок запустился, значит всё установилось верно, иначе пробуем руководство: https://stackoverflow.com/questions/78879806/docker-desktop-wsl-update-failed

1. Убедитесь, что нет ожидающих обновлений Windows.
2. При желании вы можете удалить "Docker Desktop".
3. Отключите «Подсистему Windows для Linux» в разделе «Включение и отключение компонентов Windows» (*на следующем слайде*).
4. Установите последнюю версию WSL **wsl --install**
5. Запустите **wsl --status** для подтверждения установки.
6. Установите «Docker Desktop» еще раз.

Приложение к п.3 инструкции:

Отключите «Подсистему Windows для Linux» в разделе «Включение и отключение компонентов Windows».



Когда докер запущен, вводим команду терминал:

docker-compose up –d

Терминал должен быть запущен из папки, где находится файл

docker-compose.yml

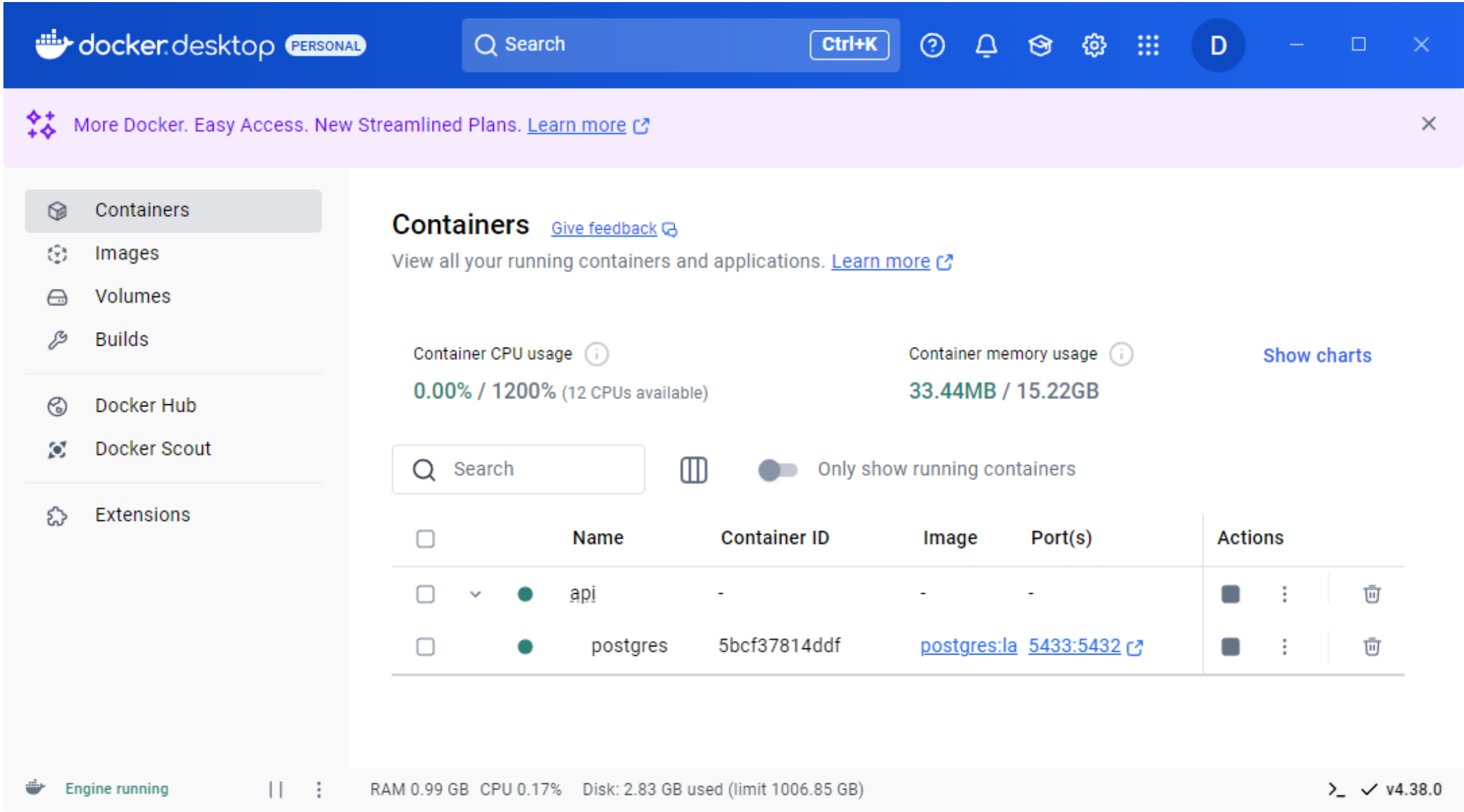
Можно ввести в терминале Visual Studio

PowerShell для разработчиков

PowerShell для разработчиков

```
**********************************************************************
** Visual Studio 2022 Developer PowerShell v17.12.4
** Copyright (c) 2022 Microsoft Corporation
**********************************************************************
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend> cd api
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend\api> docker-compose up -d
```

В результате данной команды подключится и запустится контейнер с БД PostgreSQL.



О миграциях

Модели данных могут изменяться по мере реализации функций, при добавлении или изменении новых сущностей или свойств, схемы базы данных должны быть соответствующим образом изменены для синхронизации с приложением.

Функция миграции в EF Core позволяет последовательно применять изменения схемы к базе данных, чтобы синхронизировать ее с моделью данных в приложении без потери существующих данных.

Миграции работают следующим образом.

- При изменении модели данных разработчик использует средства EF Core для добавления соответствующей миграции, описывающей обновления, необходимые для синхронизации схемы базы данных. EF Core сравнивает текущую модель с моментальным снимком старой модели, чтобы определить различия и создать исходные файлы миграции; файлы можно отслеживать в системе управления версиями проекта, как и любой другой исходный файл.
- Созданную миграцию можно применять к базе данных различными способами. EF Core записывает все примененные миграции в специальную таблицу журнала, из которой будет ясно, какие миграции были применены, а какие нет.

Далее необходимо создать миграцию.

Руководство:

https://learn.microsoft.com/ru-ru/ef/core/cli/dotnet

Необходимо открыть терминал Power Shell или терминал в Visual Studio и установить инструменты:

dotnet tool install --global dotnet-ef

Т.к. мы уже подключили все зависимости, то команда

dotnet add package Microsoft.EntityFrameworkCore.Design

не является обязательной (должна запускаться из проекта с БД, т.е. в Infrastructure/Persistence).

```
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend> cd infrastructure
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend\infrastructure> cd persistence
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend\infrastructure\persistence>
```

Проверим, что вспомогательные инструменты установились, вводим команду:

dotnet ef

В терминале должен отобразиться интерфейс

```
PS F:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend\infrastructure\persistence> dotnet ef

                    _/\__
              ---==/    \\
        ___  ___   |.    \|\
       | __|| __|  |  )   \\\
       | _| | _|   \_/ |  //|\\
       |___||_|       /   \\\/\\

Entity Framework Core .NET Command-line Tools 9.0.1

Usage: dotnet ef [options] [command]

Options:
  --version        Show version information
  -h|--help        Show help information
```

Создаём миграцию, вводим в терминал команду:

dotnet ef migrations add -p .\Infrastructure\Persistence\ -s .\Api

dotnet ef database update -p .\Infrastructure\Persistence\ -s .\Api

После параметра -p указывается путь до проекта с контекстом БД

.\Infrastructure\Persistence

После параметра -s указывается путь до проекта с точкой входа

.\Api

Может немного поругаться, но если написал build succeeded и потом Done, то нужно ввести команду ещё раз, ошибок быть не должно.

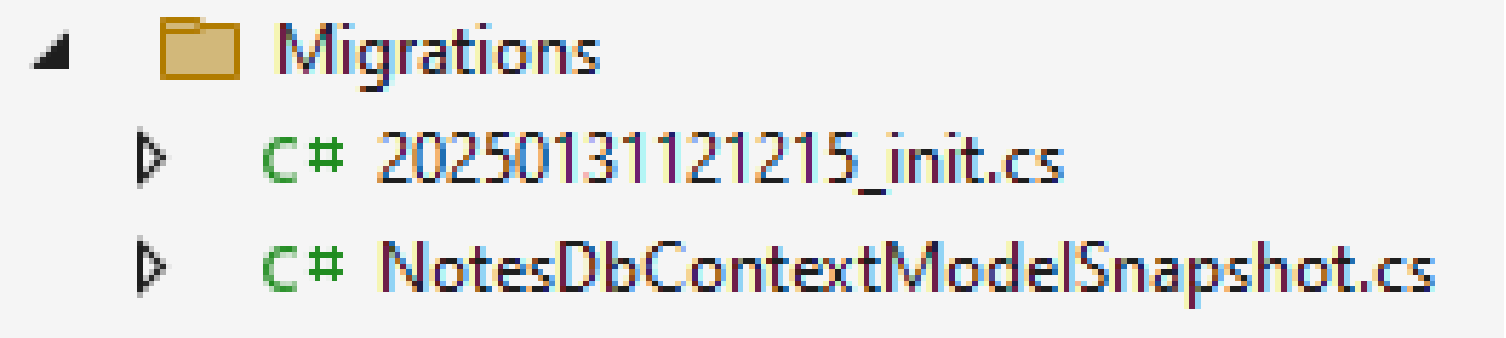
```
PS H:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend> dotnet ef database update
 -p .\Infrastructure\Persistence\ -s .\Api\
Build started...
Build succeeded.
fail: Microsoft.EntityFrameworkCore.Database.Command[20102]
      Failed executing DbCommand (20ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
Failed executing DbCommand (20ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT "MigrationId", "ProductVersion"
FROM "__EFMigrationsHistory"
ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Migrations[20411]
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lo
ck for more information if this takes too long.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for
 more information if this takes too long.
info: Microsoft.EntityFrameworkCore.Migrations[20402]
      Applying migration '20250131121215_init'.
Applying migration '20250131121215_init'.
Done.
PS H:\должность ст преп\ООП 4 сем\Примеры\Notes_fullstack_project\NotesBackend> dotnet ef database update
 init -p .\Infrastructure\Persistence\ -s .\Api\
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (15ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE IF NOT EXISTS "__EFMigrationsHistory" (
          "MigrationId" character varying(150) NOT NULL,
          "ProductVersion" character varying(32) NOT NULL,
          CONSTRAINT "PK___EFMigrationsHistory" PRIMARY KEY ("MigrationId")
      );
info: Microsoft.EntityFrameworkCore.Migrations[20411]
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lo
ck for more information if this takes too long.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for
 more information if this takes too long.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
```

База данных готова, создаем первую миграцию. Вводим команду в терминале:

dotnet ef database update init -p .\Infrastructure\Persistence\ -s .\Api

где **init** – название миграции, может быть любое.

В результате в Persistence должна создаться папка с миграциями.



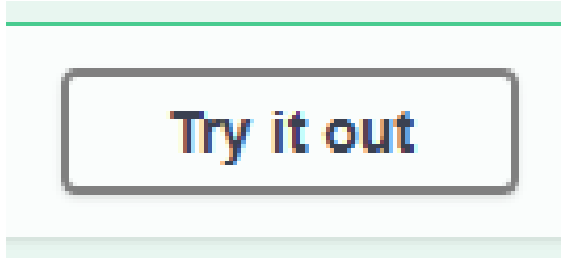
Запустим докер, контейнер с БД, после чего запустим проект.

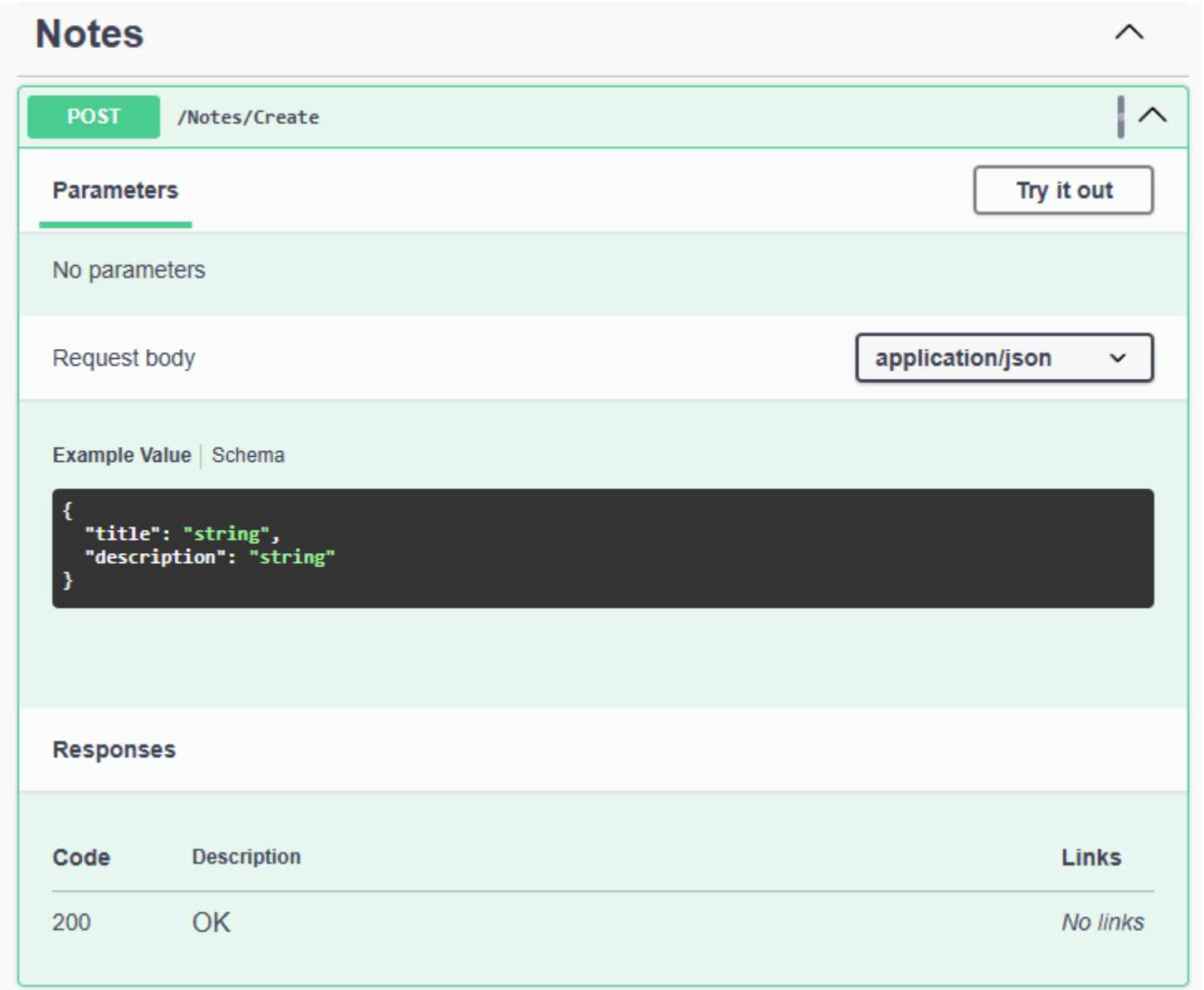
В адресной строке введем **swagger:**

https://localhost:7164/swagger/index.html

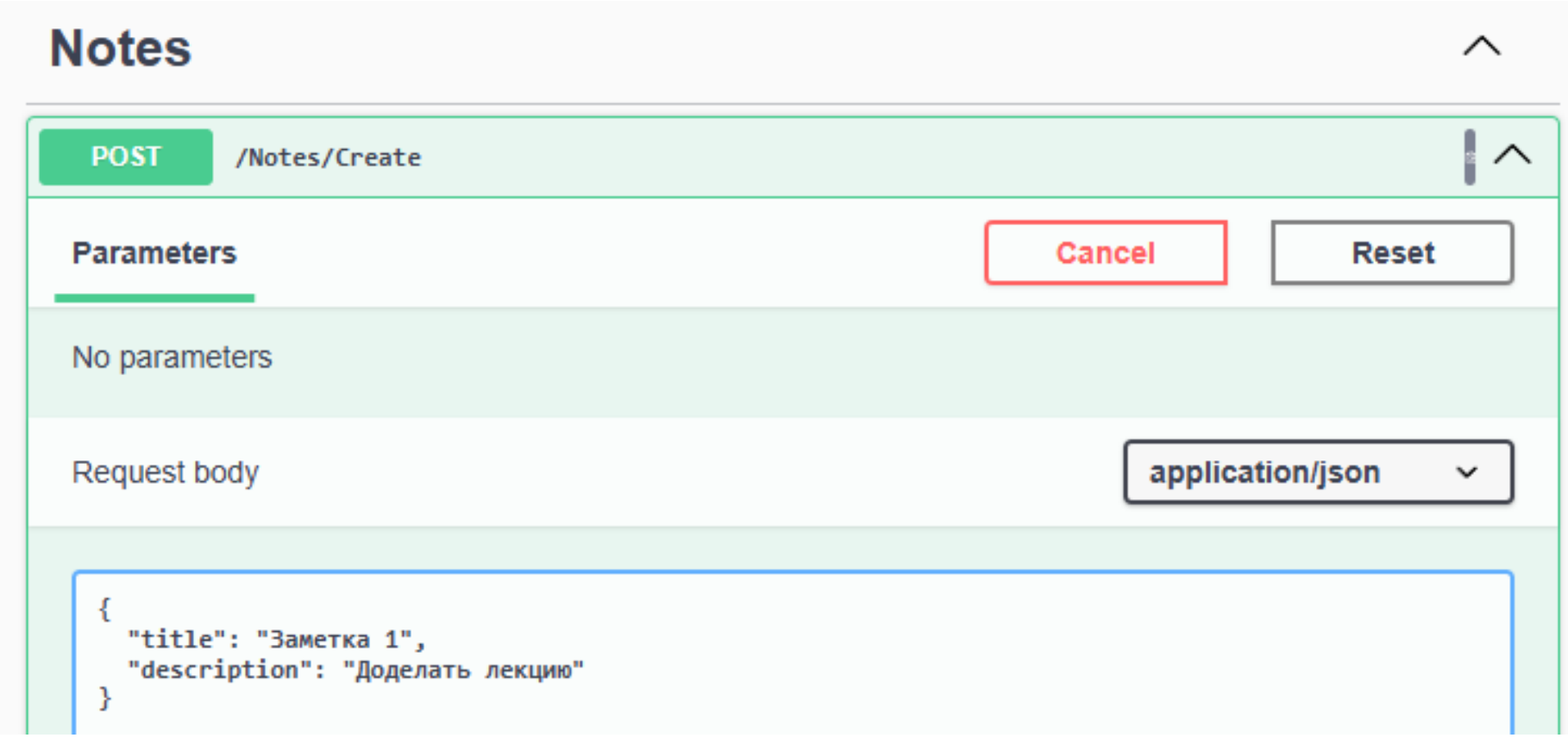
Swagger — это инструмент, который помогает разработчикам создавать, документировать и проверять API.

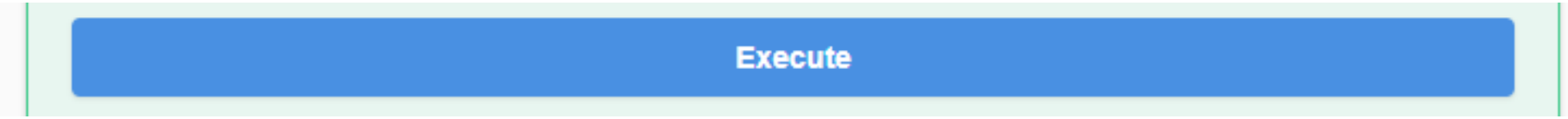
Создадим заметку:





Заполняем тело запроса и нажимаем **Execute**.





Ответ от нашего Web API.

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7164/Notes/Create' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "title": "Заметка 1",
  "description": "Доделать лекцию"
}'
```

Request URL

```
https://localhost:7164/Notes/Create
```

Server response

Ответ от нашего Web API.

Code	Details
200	

Response body

```
{
  "note": {
    "id": "1d911ad3-8da6-42f5-b105-44fa268fd6c2",
    "title": "Заметка 1",
    "description": "Доделать лекцию",
    "createdAt": "2025-02-02T15:45:21.1305074Z"
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun,02 Feb 2025 15:45:21 GMT
server: Kestrel
```

Responses

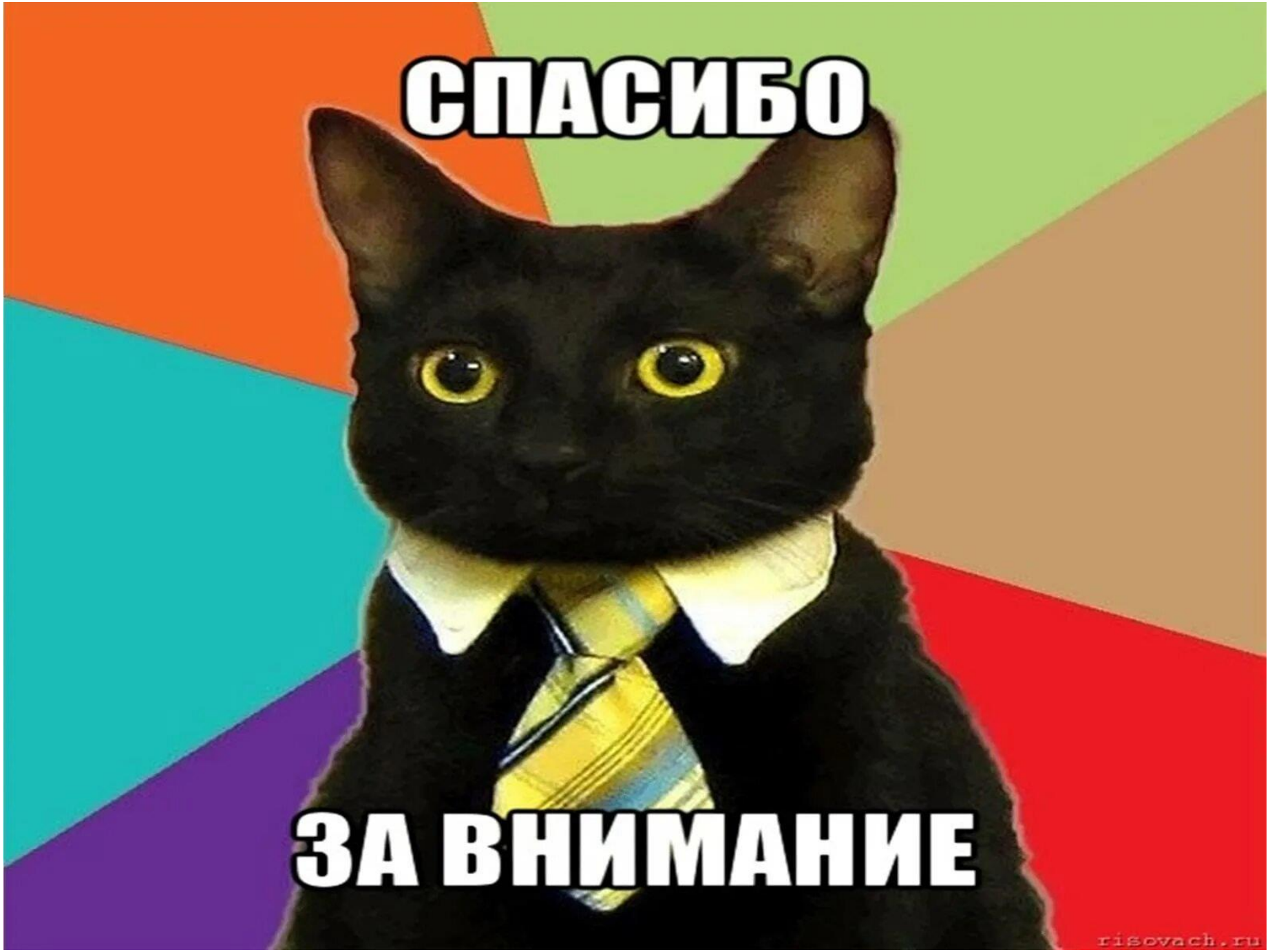
Code	Description	Links
200	OK	*No links*

Приложение

Эталонное приложение eShopOnWeb использует подход на основе чистой архитектуры для организации кода в проекты. Шаблон решения, который можно использовать в качестве отправной точки для собственных решений ASP.NET Core, находится в репозитории GitHub ardalis/cleanarchitecture. Кроме того, вы можете установить шаблон из NuGet.

Осталось за кадром

1. Аутентификация и авторизация (JWT).
2. Global Error Handling – отлов и обработка ошибок запросов.
3. CQRS – подход для больших, масштабируемых приложений.
4. MediatR – фреймворк, реализующий CQRS.
5. SignalR – библиотека, в которой реализована двунаправленная связь для обмена сообщениями между клиентом и сервером.
6. Mapster – фреймворк для автоматического сопоставления объектов между моделями.
7. Domain Driven Design (DDD) – подход к проектированию больших, масштабируемых приложений.
8. Взаимодействие с клиентом, например, на React JS.


СПАСИБО
ЗА ВНИМАНИЕ
risovach.ru